But you can't do this:

```
iex> "text" <> symbol <> number = "text:7"
** (ArgumentError) the left argument of <> operator inside a match
   should always be a literal binary because its size can't be
   verified. Got: symbol
```

Even with this limitation, it's still extremely useful.

Let's explore other powerful pattern-matching forms. We'll look at lists, maps, and tuples next.

Match Data Structures

Lists, tuples, and maps are fully compatible with pattern matching. You'll commonly use this in two ways. The first is to extract data structure components into variables so you can operate on them. The second is to check if an input matches a certain structure, as part of control flow.

This section focuses on extracting the components of data structures. Open a new IEx session and type the following:

```
$ iex
iex> [a] = [1]
iex> a
1

iex> {:ok, result} = {:ok, "my result"}
iex> result
"my result"

iex> [a, 2, c] = [1, 2, 3]
iex> {a, c}
{1, 3}

iex> [a] = [1, 2]
** (MatchError) no match of right hand side value: [1, 2]
```

Lists and tuples can be matched on an exact-position basis. Each position on the right and left must have compatible patterns. You can even separate a data structure into multiple variables, like `a` and `c` in the previous code. If your structure doesn't match the pattern provided, you get a MatchError.

Parts of a list are matched with the | and ++ operators:

```
iex> [head | tail] = [1, 2, 3, 4]
iex> head
1

iex> tail
[2, 3, 4]
```